Food Waste Marketplace

Technical Design Document

DubHacks '26 · Version 1.0 · October 17, 2026

> Stores list surplus food at a discount near closing time. Buyers reserve and pay in-app. Sellers are paid out instantly, and every pickup is logged as meals saved.

Audience: the four-person build team (A seller flow, B buyer flow, C payments, D backend and data), plus judges and mentors who want to understand how the system works.

Status: design baseline for the hackathon build. Items marked *to confirm* depend on sponsor documentation or the hacker guide and have not been verified.

Contents

1. Summary **5**

1.1 Goals 5

1.2 Non-goals 5

1.3 Track mapping 5

2. Users and flows **6**

2.1 Personas 6

2.2 Seller flow 6

2.3 Buyer flow 6

3. Architecture **6**

3.1 Components 7

3.2 Key design decisions 7

4. Data model **8**

4.1 Tables 8

4.2 Schema (excerpt) 8

4.3 Reservation lifecycle 9

4.4 Impact formulas 9

5. API reference **9**

5.1 Example: create a listing draft 10

5.2 Example: pay for a reservation 10

5.3 Real-time events (SSE) 10

5.4 Errors . . . 11

6. AI pipeline **11**

6.1 Steps . . . 11

6.2 Pricing rule . . . 11

6.3 Prompt (Gemini) . . . 12

7. Integration specifications **12**

7.1 Perceptron (vision) . . . 12

7.2 Gemini (listing and pricing) . . . 12

7.3 CloudForce nebulaONE . . . 12

7.4 Backboard (buyer memory) . . . 13

7.5 T-Mobile (real-time alerts) . . . 13

7.6 Visa (checkout) . . . 13

7.7 Solana (payouts) . . . 13

7.8 Tiger Data . . . 13

8. Front end **14**

8.1 Routes . . . 14

8.2 PWA requirements . . . 14

8.3 Design guidelines (Design = 20%) . . . 14

9. Security and privacy **14**

10. Repository, configuration and deployment **15**

10.1 Repository layout . . . 15

10.2 Environment variables . . . 15

10.3 Deployment on Vultr . . . 15

10.4 Local development . . . 16

11. Testing and demo readiness **16**

11.1 Test plan . . . 16

11.2 Seed data . . . 16

11.3 Readiness checklist . . . 16

12. Team plan and timeline **17**

12.1 Ownership . . . 17

12.2 Timeline . . . 17

12.3 Risks . . . 17

13. Demo script and pitch **17**

13.1 Two-minute demo . . . 18

13.2 Startup framing (DubHacks Next / Commit) . . . 18

14. Open questions **18**

1. Summary

The product is a two-sided marketplace for surplus food. A seller photographs leftovers near closing; vision and language models turn the photo into a priced listing in seconds. Nearby buyers who match the listing get an alert, reserve, and pay in the app. The seller receives a Solana payout within seconds, and each completed pickup is recorded as meals saved, kilograms diverted and dollars recovered.

Technically, it is a single Next.js application (PWA front end plus API routes) deployed on Vultr, backed by one Tiger Data (Postgres + TimescaleDB) database, with a background worker for payouts and timers. Every sponsor service sits behind an adapter with a live and a mock implementation, so the demo still works if any one sandbox fails.

1.1 Goals

- Run the full core loop end to end on the live domain from two phones: photo to listing to alert to payment to payout to impact.
- Use each sponsor technology in a role the judges can see in the demo, not as a footnote.
- Score on all four judging criteria: Functionality 30%, Track Applicability 30%, Design 20%, Creativity 20%.
- Keep the system small enough for four people to build in under 24 hours.

1.2 Non-goals

- Native iOS or Android builds; the PWA covers install, camera and push.
- Real money movement: Visa runs in sandbox and Solana on devnet.
- Delivery logistics, ratings, refunds and dispute handling.
- Multi-region scaling or high availability beyond one Vultr instance.

1.3 Track mapping

Track	Where it appears in the product	Visible in demo
Visa (title)	Checkout: buyer reserves and pays in-app	Step 3
T-Mobile	Real-time nearby-deal alerts on the phone (Web Push + SSE); requirement to confirm	Step 3
Perceptron	Identifies items in the seller's photo	Step 2
Gemini	Writes listing copy and proposes the discount	Step 2
CloudForce (nebulaONE)	Extra pipeline step (allergen tags, tone check); shareable impact cards for Most Viral	Steps 2, 5
Backboard	Remembers buyer preferences to target alerts	Step 3
Solana	Instant low-fee payout to the store wallet	Step 4
Tiger Data	Single database; hypertable for impact metrics	Step 5
Vultr	Hosts web app, API and worker	Implicit
GoDaddy	Project domain with HTTPS	URL on screen
DubHacks Next / Commit	Startup pitch: market, model, traction plan (section 13)	Pitch

Track	Where it appears in the product	Visible in demo
DEV Writing	Post-event write-up drawn from this document	After event

2. Users and flows

2.1 Personas

Persona	Needs	Key screens
Seller (bakery, café, small restaurant)	List leftovers in under 30 s, near closing, with no pricing guesswork; get paid fast	/sell, /sell/review, /orders, /impact
Buyer (student, commuter)	Cheap food nearby, alerts only for things they want, quick pickup	/deals, /deals/:id, /checkout, /me
Judge / viewer	See the whole loop and the impact in 2 minutes	/impact, shared impact card

2.2 Seller flow

1. Open /sell and take a photo with the device camera (*input type=file capture=environment*).
2. The app uploads it to *POST /listings/draft*. Perceptron returns items; Gemini returns title, description, tags and a discount.
3. The review screen shows the draft with editable fields: title, quantity, original price, discounted price, pickup window.
4. Publish calls *POST /listings*. The listing goes live and alerts fan out.
5. When a buyer pays, the seller sees the order and, seconds later, the Solana payout with an explorer link.
6. At pickup, the seller scans the buyer's QR code; the order completes and the impact counters rise.

2.3 Buyer flow

1. First visit: grant location and notifications; set preferences (diet, max price, radius, favourite stores). Saved to Backboard.
2. A push notification or live feed card announces a nearby match.
3. Listing detail shows photo, price, savings, distance and time left to pick up.
4. Reserve holds the quantity for 10 minutes; checkout runs the Visa sandbox payment.
5. Confirmation shows a pickup QR code and directions. Picking up closes the loop.

3. Architecture

Clients talk only to the API. The API owns all sponsor credentials, writes to Tiger Data, and pushes real-time events to clients. The worker runs anything that should not block a request: payouts, expiry timers and alert fan-out.

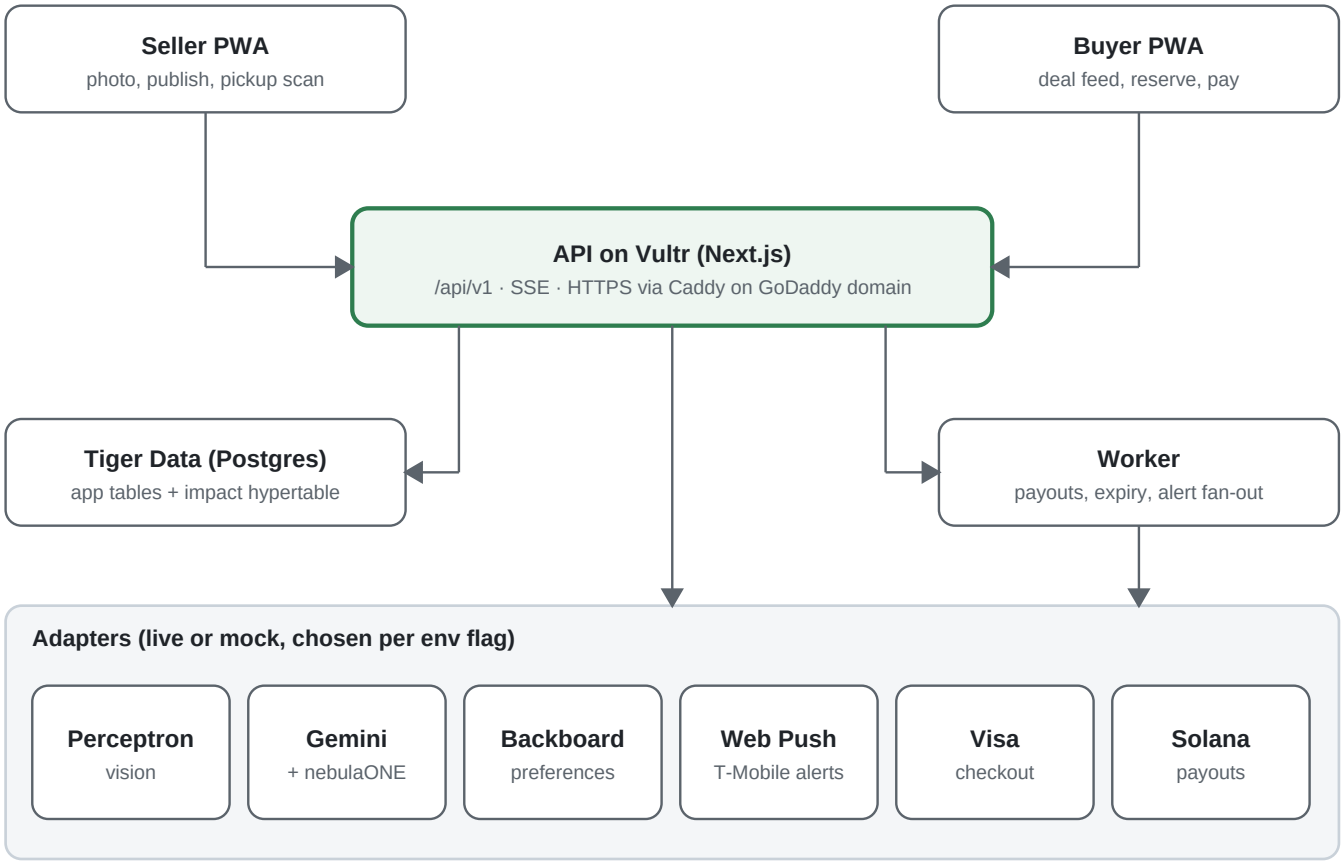


Figure 1. System architecture.

3.1 Components

Component	Technology	Responsibility	Owner
PWA client	Next.js (App Router), React, Tailwind	Seller and buyer UIs, camera, service worker, push subscription	A, B, D
API	Next.js route handlers, Node 20, TypeScript	Validation, auth, orchestration of adapters, SSE stream	D
Worker	Node process, polling a jobs table	Solana payouts, reservation holds, listing expiry, alert fan-out	C, D
Database	Tiger Data (Postgres + TimescaleDB)	App data, job queue, impact hypertable and aggregates	D
Reverse proxy	Caddy	HTTPS certificates for the GoDaddy domain, routing	D
Adapters	TypeScript modules	One per sponsor, live + mock	A, B, C

3.2 Key design decisions

Decision	Why	Trade-off
One Next.js app for UI and API	One deploy, one repo, shared types	Less separation; fine at hackathon scale

Decision	Why	Trade-off
PWA instead of native	No app-store builds; camera and push work in the browser	iOS push needs the app added to the home screen (iOS 16.4+)
One database for app data and metrics	One thing to run; Tiger Data covers both	Analytics share load with the app; negligible here
Jobs table instead of a queue service	No extra infrastructure; *FOR UPDATE SKIP LOCKED* is enough	Polling latency of about 1 s
SSE instead of WebSockets	Works through Caddy with no extra setup; one-way is all we need	Client actions still go over REST
Adapter pattern with mocks	Demo survives any single sandbox outage	Extra code per integration
Server-clamped AI pricing	A bad generation can never list food for $0	Less freedom for the model

4. Data model

All money is stored as integer cents (USD) and all times as *timestamptz* in UTC. IDs are UUIDs generated in the database. Status columns use Postgres enums so invalid states are rejected at write time.

4.1 Tables

Table	Columns	Notes
stores	id, name, address, lat, lng, closes_at (time), timezone, solana_wallet, visa_merchant_ref, created_at	One row per seller location
users	id, role (buyer | seller), name, email, phone, store_id (nullable), backboard_user_id, created_at	Seller rows link to a store
push_subscriptions	id, user_id, endpoint, p256dh, auth, created_at	Web Push targets
listings	id, store_id, title, description, items (jsonb), tags (text[]), photo_url, original_cents, price_cents, discount_pct, qty_total, qty_available, pickup_until, status, ai_meta (jsonb), created_at	ai_meta keeps raw model output for debugging
reservations	id, listing_id, buyer_id, qty, total_cents, status, hold_expires_at, pickup_code, idempotency_key, created_at, paid_at, picked_up_at	Unique (buyer_id, idempotency_key)
payments	id, reservation_id, visa_txn_id, visa_status, payout_cents, solana_sig, payout_status, created_at	One per reservation
jobs	id, kind, payload (jsonb), run_at, attempts, last_error, done_at	Worker queue
impact_events	ts, store_id, reservation_id, meals_saved, kg_saved, co2e_kg, cents_recovered	Hypertable on ts

4.2 Schema (excerpt)

```
CREATE TYPE listing_status AS ENUM ('active','sold_out','expired','withdrawn');
CREATE TYPE reservation_status AS ENUM ('pending','paid','picked_up','cancelled');

CREATE TABLE listings (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  store_id       uuid NOT NULL REFERENCES stores(id),
  title          text NOT NULL,
  description    text NOT NULL,
  items          jsonb NOT NULL,           -- [{name, qty, confidence}]
  original_cents int  NOT NULL CHECK (original_cents > 0),
  price_cents    int  NOT NULL CHECK (price_cents > 0 AND price_cents <= original_cents),
  qty_total      int  NOT NULL CHECK (qty_total > 0),
```

```
  qty_available  int  NOT NULL CHECK (qty_available >= 0),
  pickup_until   timestamptz NOT NULL,
  status         listing_status NOT NULL DEFAULT 'active',
  created_at     timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX listings_active_geo ON listings (status, pickup_until) WHERE status = 'active';

CREATE TABLE impact_events (
  ts timestamptz NOT NULL, store_id uuid NOT NULL, reservation_id uuid,
  meals_saved int NOT NULL, kg_saved numeric NOT NULL,
  co2e_kg numeric NOT NULL, cents_recovered int NOT NULL
);
SELECT create_hypertable('impact_events', 'ts');

CREATE MATERIALIZED VIEW impact_daily WITH (timescaledb.continuous) AS
  SELECT time_bucket('1 day', ts) AS day, store_id,
         sum(meals_saved) AS meals, sum(kg_saved) AS kg,
         sum(co2e_kg) AS co2e, sum(cents_recovered) AS cents
  FROM impact_events GROUP BY day, store_id;
```

4.3 Reservation lifecycle

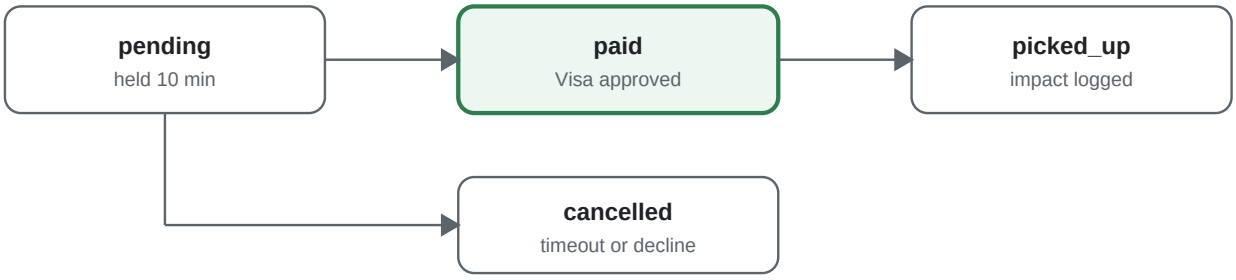


Figure 2. Reservation states. Only paid reservations trigger a payout.

A *pending* reservation decrements *qty_available* immediately so two buyers cannot take the last item. If it is not paid before *hold_expires_at* (10 minutes), the worker cancels it and returns the quantity. A listing becomes *sold_out* when *qty_available* reaches 0 and *expired* at *pickup_until*.

4.4 Impact formulas

Constants are demo assumptions to be cited or adjusted before the pitch; they are labelled as estimates in the UI.

Metric	Formula	Default assumption
meals_saved	sum of item quantities in the reservation	1 item = 1 meal-equivalent
kg_saved	meals_saved × kg per meal	0.4 kg per item (estimate)
co2e_kg	kg_saved × emission factor	2.5 kg CO2e per kg food (estimate, to source)
cents_recovered	total paid by buyer	Revenue the store would otherwise lose

5. API reference

Base path */api/v1*. JSON in and out except the photo upload (multipart). Authentication for the demo is a signed session cookie issued by a simple login that selects a seeded user; production would use OAuth or magic links.

Method	Path	Owner	Purpose
POST	/auth/demo-login	D	Pick a seeded buyer or seller; sets session cookie

Method	Path	Owner	Purpose
POST	/listings/draft	A	Photo → Perceptron → Gemini; returns an unsaved draft
POST	/listings	A	Publish a confirmed draft; enqueue alert fan-out
PATCH	/listings/:id	A	Edit price or quantity; withdraw
GET	/listings/nearby	B	Active listings near lat/lng, ranked by preference fit, distance, time left
GET	/listings/:id	B	Listing detail with store info
GET / PUT	/me/preferences	B	Read or write Backboard memory
POST	/push/subscribe	B	Store a Web Push subscription
POST	/reservations	C	Create a 10-minute hold
POST	/reservations/:id/pay	C	Visa sandbox charge; on success enqueue payout
POST	/reservations/:id/cancel	C	Release hold
POST	/reservations/:id/pickup	D	Validate pickup code; log impact
GET	/orders	C	Seller's reservations and payout status
GET	/impact/summary	D	Totals + daily series, platform-wide or per store
GET	/impact/card/:storeId.png	D	Shareable impact image (Open Graph size)
GET	/stream	B	Server-Sent Events channel
GET	/health	D	Live or mock status per adapter, DB connectivity

5.1 Example: create a listing draft

```
POST /api/v1/listings/draft          (multipart: photo, store_id)

200 OK
{
  "items": [{"name": "croissant", "qty": 6, "confidence": 0.93},
            {"name": "blueberry muffin", "qty": 4, "confidence": 0.88}],
  "title": "Evening pastry box: 6 croissants + 4 muffins",
  "description": "Baked this morning, still flaky. Great for tomorrow's breakfast.",
  "tags": ["vegetarian", "contains-gluten", "contains-dairy"],
  "original_cents": 4200,
  "discount_pct": 55,
  "price_cents": 1890,
  "pickup_until": "2026-10-18T03:00:00Z",
  "source": {"vision": "live", "llm": "live"}
}
```

5.2 Example: pay for a reservation

```
POST /api/v1/reservations/7f3c.../pay
Idempotency-Key: 2b9e5c1a-...
{ "payment_token": "<from Visa sandbox checkout>" }

200 OK
{ "status": "paid", "visa_txn_id": "sbx_81kd...", "payout": {"status": "queued"} }
```

5.3 Real-time events (SSE)

Event	Sent to	Payload
listing.new	Buyers in radius	listing summary, distance

Event	Sent to	Payload
listing.updated	Viewers of the listing	id, qty_available, status
reservation.paid	Seller	reservation id, qty, total
payout.confirmed	Seller	solana_sig, amount, explorer URL
impact.updated	Dashboard viewers	new totals

5.4 Errors

All errors return *{"error": {"code", "message"}}* with an HTTP status.

HTTP	Code	When
400	invalid_input	Validation failed (zod schema)
401	unauthenticated	No session
403	forbidden	Seller acting on another store's listing
404	not_found	Unknown id
409	sold_out	Not enough quantity left
409	hold_expired	Paying after the 10-minute hold
402	payment_declined	Visa declined
502	upstream_failed	Sponsor API error; response says whether a mock was used

6. AI pipeline

6.1 Steps

1. **Vision (Perceptron).** The photo is resized to 1280 px on the long side and sent for item detection. Output: item names, counts, confidence.
2. **Normalise.** Items below 0.5 confidence are dropped; synonyms merge ("croissants" and "croissant").
3. **Listing and price (Gemini).** A JSON-mode prompt gets title, description, tags and discount, given items, store type, local time and minutes to closing.
4. **Check (nebulaONE, CloudForce).** Second pass for allergen tags and tone. Optional; skipped if unavailable. Requirement to confirm.
5. **Clamp.** The server bounds the discount and computes the final price. The seller can still edit everything.

6.2 Pricing rule

The model proposes; the server decides. The discount floor rises as closing nears, and the model may move up to 10 points above that floor based on item type (more for perishables such as sandwiches, less for bread).

Minutes to closing	Floor	Ceiling
More than 180	20%	40%
60 to 180	30%	50%
20 to 60	50%	65%

Minutes to closing	Floor	Ceiling
Under 20	60%	75%

6.3 Prompt (Gemini)

```
System: You write short, appetizing listings for surplus food sold at a discount.
Return JSON only: {title, description, tags[], discount_pct}.
Title max 60 chars. Description max 160 chars, no exaggeration, no health claims.
Tags from: vegetarian, vegan, contains-gluten, contains-dairy, contains-nuts.
discount_pct must be between {floor} and {ceiling}.

User: Store: {store_name} ({store_type}). Local time {time}, closes in {minutes} min.
Items: {items_json}. Original total price: ${original}.
```

Responses that fail JSON parsing or schema validation are retried once, then replaced by the template fallback.

7. Integration specifications

Every adapter exports the same interface for its live and mock versions. *lib/adapters/index.ts* picks the implementation from *MOCK_<NAME>* and falls back to the mock automatically if the live call throws and *AUTO_FALLBACK=1*. Fallbacks are logged and surfaced on */health*.

```
export interface PayAdapter {
  charge(r: { reservationId: string; amountCents: number; token: string; idempotencyKey: string })
    : Promise<{ ok: boolean; txnId: string; status: 'approved' | 'declined' }>;
}
export interface PayoutAdapter {
  send(p: { wallet: string; amountCents: number; reservationId: string })
    : Promise<{ signature: string; explorerUrl: string }>;
}
```

7.1 Perceptron (vision)

Item detection from the seller's photo.

- Input: JPEG under 2 MB. Output: items with counts and confidence.
- Timeout 6 s; on timeout use the mock.
- Mock: returns the items for the bundled demo photos by file hash, else a generic "assorted baked goods".

Fallback / notes: Exact API and track requirements to confirm in the hacker guide.

7.2 Gemini (listing and pricing)

Generates listing text and a discount proposal.

- JSON-mode output validated with zod.
- Store raw response in listings.ai_meta.
- Timeout 8 s, one retry.

Fallback / notes: Mock: template title from top two items; discount at the midpoint of the band.

7.3 CloudForce nebulaONE

Optional checking step in the AI pipeline, and the impact-card generator for the Most Viral prize.

- Impact card: 1200×630 PNG with store name, meals saved, dollars recovered, project URL.
- Share via Web Share API on phones; copy-link fallback on desktop.

Fallback / notes: Requirements to confirm in the hacker guide.

7.4 Backboard (buyer memory)

Persistent memory of buyer preferences and behaviour.

- Writes: explicit preferences, plus implicit signals (stores bought from, categories reserved, times of day).
- Reads: at alert fan-out and feed ranking.
- Keyed by users.backboard_user_id.

Fallback / notes: Mock: preferences in a jsonb column on users.

7.5 T-Mobile (real-time alerts)

Nearby-deal alerts delivered to the phone.

- Web Push with VAPID keys through the service worker; SSE for open apps.
- Fan-out: buyers whose last location is within their radius and whose preferences match; at most 3 alerts per buyer per hour.
- Use any specific T-Mobile API the track requires (to confirm).

Fallback / notes: Mock: SSE only, plus an in-app banner.

7.6 Visa (checkout)

Buyer payment in sandbox.

- Client collects payment details in the sponsor's hosted or tokenised flow; the server never sees raw card numbers.
- Server charges with the token and an idempotency key.
- Sandbox setup (credentials, certificates) starts first thing; it is the top schedule risk.

Fallback / notes: Mock: approves after 800 ms; card ending 0002 declines to demo the error path.

7.7 Solana (payouts)

Instant, low-fee payout to the store wallet.

- Devnet; platform wallet funded by airdrop before the demo.
- Payout = buyer total minus a 10% platform fee (demo assumption).
- Convert cents to SOL at a fixed demo rate, or send devnet USDC (decision open).
- Worker confirms the transaction, stores the signature, emits payout.confirmed.

Fallback / notes: Mock: fake signature after a 2 s delay; explorer link hidden.

7.8 Tiger Data

Database for everything.

- Managed service; connection pool size 10.
- Migrations in lib/db/migrations, run on deploy.

Fallback / notes: Fallback: local Postgres with the TimescaleDB extension in Docker.

8. Front end

8.1 Routes

Route	Who	Content
/	All	Pitch landing, live impact counter, "Find deals" and "I'm a store"
/deals	Buyer	Map toggle + list of nearby deals, live updates
/deals/:id	Buyer	Photo, price vs original, distance, countdown, Reserve
/checkout/:reservationId	Buyer	Hold countdown, Visa payment, confirmation with QR
/me	Buyer	Preferences, notification settings, past pickups, personal impact
/sell	Seller	Camera capture, recent listings
/sell/review	Seller	Editable AI draft, Publish
/orders	Seller	Incoming orders, payout status, QR scanner
/impact	All	Dashboard: meals, kg, CO2e, dollars recovered, daily chart, per-store leaderboard

8.2 PWA requirements

- manifest.json with name, icons (192, 512), theme colour, display standalone.
- Service worker: caches the app shell, handles push events, opens the listing on notification click.
- HTTPS on the real domain is required for service workers, camera and push.
- iOS: push works only after Add to Home Screen; show an install hint on Safari.

8.3 Design guidelines (Design = 20%)

- Mobile first at 390 px wide; one primary action per screen, thumb-reachable.
- One accent colour (green) for savings and success; neutral greys elsewhere.
- Show savings prominently: strike-through original price, discounted price, percent off.
- Countdowns for pickup windows and holds create urgency without clutter.
- Real food photos in seed data; no stock placeholders in the demo path.
- Skeleton loaders for AI steps so the 3 to 5 s wait feels intentional.

9. Security and privacy

Area	Measure
Secrets	All sponsor keys server-side in env vars; .env never committed; shared via a password vault
Payments	No raw card data touches our server; tokenised sandbox flow; idempotency keys on charge
Wallets	Platform devnet key only on the server; store wallets are public addresses
Auth	HttpOnly, Secure, SameSite=Lax session cookie; role checks on every seller route

Area	Measure
Location	Buyer location used for the query and stored coarsely (about 1 km); not shared with sellers
Uploads	Type and size checked (JPEG/PNG, under 5 MB); EXIF stripped before storage
AI output	Schema-validated; prices clamped server-side; seller reviews before publishing
Rate limits	Per-IP limits on draft creation and payment endpoints

10. Repository, configuration and deployment

10.1 Repository layout

```
/app                 Next.js routes: /, /deals, /sell, /checkout, /orders, /impact, /me
/app/api/v1          route handlers
/components          UI components
/lib/adapters        vision, llm, check, memory, notify, pay, payout  (live.ts + mock.ts each)
/lib/db              client, migrations/, seed.ts
/lib/pricing.ts      discount bands and clamp
/lib/impact.ts       impact formulas
/worker              index.ts: job loop (payout, hold-expiry, listing-expiry, fanout)
/public              manifest.json, sw.js, icons, demo photos
docker-compose.yml   web, worker, caddy
Caddyfile
.env.example
```

10.2 Environment variables

Variable	Purpose
DATABASE_URL	Tiger Data connection string
SESSION_SECRET	Cookie signing
PERCEPTRON_API_KEY	Vision
GEMINI_API_KEY	Listing copy and pricing
NEBULAONE_*	CloudForce pipeline (to confirm)
BACKBOARD_API_KEY	Buyer memory
VISA_*	Sandbox credentials and certificate paths
SOLANA_RPC_URL, PLATFORM_WALLET_SECRET	Devnet payouts
VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY	Web Push
MOCK_VISION, MOCK_LLM, MOCK_MEMORY, MOCK_NOTIFY, MOCK_PAY, MOCK_PAYOUT	Per-adapter mock switches
AUTO_FALLBACK	Fall back to mock when a live call fails
PUBLIC_BASE_URL	Domain used in links and impact cards

10.3 Deployment on Vultr

1. Create an Ubuntu 24.04 instance (2 vCPU, 4 GB is plenty) and install Docker.
2. Register the domain on GoDaddy early; point an A record at the instance IP (DNS can take a while).
3. Clone the repo, copy .env, run *docker compose up -d --build*.

4. Caddy requests TLS certificates automatically once DNS resolves.
5. Run migrations and the seed script: *docker compose exec web npm run db:migrate && npm run db:seed*.
6. Check */api/v1/health* shows the expected live and mock adapters.

```
# Caddyfile
yourdomain.example {
    reverse_proxy web:3000
}
```

10.4 Local development

```
cp .env.example .env          # set MOCK_*=1 to run with no sponsor keys
docker compose up db -d       # local Postgres + TimescaleDB
npm install && npm run db:migrate && npm run db:seed
npm run dev                   # web on :3000
npm run worker                # job loop
```

11. Testing and demo readiness

11.1 Test plan

Level	What	How
Unit	Pricing clamp, impact formulas, schema validation	Vitest
Adapter contract	Live and mock return the same shapes	One test per adapter, run with MOCK on and off
End to end	Full core loop	Scripted run on two phones against the live domain
Concurrency	Two buyers reserve the last item	Parallel requests; exactly one succeeds
Failure	Each sponsor down	Flip AUTO_FALLBACK, kill the key, confirm demo continues

11.2 Seed data

- One hero bakery for the demo, with real photos of pastries.
- About 10 nearby stores around campus with varied closing times and listings.
- Fifteen demo buyers with different preferences in Backboard.
- Two weeks of impact_events so the dashboard shows a trend on day one.

11.3 Readiness checklist

- ■ Devpost draft submitted by 10:30 PM with all members, Discords and tracks declared
- ■ Full loop works on the live domain from two phones
- ■ Push notification arrives on the buyer phone within 5 s of publish
- ■ Solana payout confirms on stage within about 10 s
- ■ Every adapter tested in mock mode
- ■ Impact card renders and shares
- ■ Backup demo video recorded

- T-Mobile, Perceptron and CloudForce requirements confirmed
- Final submission before 10 AM

12. Team plan and timeline

12.1 Ownership

Person	Owns	Done by 10 PM means
A	Seller flow, vision + LLM + nebulaONE adapters, pricing	Photo in, editable draft out, live and mock
B	Buyer feed, preferences, Backboard, push and SSE	Feed shows seed listings; push reaches a second phone
C	Visa and Solana adapters, reservations, worker payouts	One sandbox charge and one devnet payout succeed
D	API skeleton, schema, impact dashboard, deploy, domain, design polish	App on the domain over HTTPS; dashboard reads seed data

12.2 Timeline

When	What	Exit criterion
Now to 5 PM	Repo, hello-world on Vultr, all API keys, buy domain	Deployed page loads; keys in vault
5 to 10 PM	Each person's piece works alone	Each adapter passes its contract test
By 10:30 PM	Devpost draft	Submitted
Overnight	Connect the full loop	End-to-end run succeeds
6 to 9 AM	UI polish, seed data, backup video	Video saved in two places
Before 10 AM	Final submission	Submitted
After event	DEV post, Commit Fellowship form	Published / sent

12.3 Risks

Risk	Likelihood	Mitigation
Visa sandbox setup takes hours	High	Person C starts immediately; mock ready from hour one
Solana devnet congestion or airdrop limits	Medium	Pre-fund wallet early; payout mock with AUTO_FALLBACK
iOS push restrictions on stage phone	Medium	Use Android for the buyer phone, or install PWA beforehand
DNS not propagated	Medium	Buy domain first; Vultr IP as fallback
Venue Wi-Fi unreliable	Medium	Phone hotspot; backup video
Model output poor on stage photo	Low	Rehearse with the exact photo; seller can edit
Track requirements misunderstood	Medium	Confirm in hacker guide before 5 PM

13. Demo script and pitch

13.1 Two-minute demo

Time	Beat	On screen
0:00	Problem: how much a nearby bakery throws out each night	Landing page, live counter
0:20	Seller snaps leftover pastries; listing appears with a price	/sell → /sell/review
0:50	Switch phones: alert arrives; reserve and pay with Visa	Notification → /checkout
1:20	Seller's Solana payout lands within seconds	/orders with explorer link
1:40	Impact: meals saved, dollars recovered, shareable card	/impact

13.2 Startup framing (DubHacks Next / Commit)

- **Customer:** independent bakeries, cafés and small restaurants that discard unsold food daily.
- **Value:** recover revenue on food that would be thrown out, with listing in under 30 seconds and payouts in seconds.
- **Model:** commission on each sale (10% assumed for the demo).
- **Wedge:** campus neighbourhoods with dense student demand near closing time.
- **Edge:** AI listing and pricing removes the seller's effort; instant payouts suit small businesses' cash flow.
- Market size and competitor figures are not included here and should be researched and cited before pitching.

14. Open questions

- Which Visa sandbox product and checkout flow fits the title track best?
- Payout in devnet SOL or devnet USDC?
- What exactly do the T-Mobile, Perceptron and CloudForce tracks require?
- Emission and kg-per-meal factors: which sources to cite?
- Platform fee for the pitch: 10% or another figure?